

CURSO

TEST AUTOMATION ENGINEER

FORMACIÓN INTEGRAL



Objetivo General del Curso

ANALIZAR LAS HERRAMIENTAS DE DISEÑO DE TEST AUTOMATION ENGINEER, DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Objetivo específico del Módulo

IDENTIFICAR LOS FUNDAMENTOS DEL TESTING DE SOFTWARE,
DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Contenidos

Objetivo General del Curso	2
Objetivo específico del Módulo.....	2
Módulo 1: FUNDAMENTOS DEL TESTING DE SOFTWARE.....	4
Capítulo 1: ROL DEL QA AUTOMATION EN EQUIPOS ÁGILES.....	6
¿Qué hace un QA Automation en un equipo ágil?	6
Beneficios del QA Automation en entornos ágiles	7
Habilidades de un QA Automation en un equipo ágil.....	7
Capítulo 2: TIPOS DE PRUEBAS: UNITARIAS, INTEGRACIÓN, REGRESIÓN, SMOKE, SANITY	9
1. Pruebas Unitarias (Unit Tests)	10
2. Pruebas de Integración (Integration Tests)	11
3. Pruebas de Regresión (Regression Tests)	12
4. Pruebas Smoke (Smoke Tests)	13
5. Pruebas Sanity (Sanity Tests)	14
Resumen comparativo.....	15
Capítulo 3: CICLO DE VIDA DEL BUG Y REPORTE DE INCIDENCIAS	16
Ciclo de vida del Bug (Bug Life Cycle).....	17
Reporte de Incidencias	18
Herramientas comunes para gestión de bugs e incidencias:.....	19
Buenas prácticas en el reporte de incidencias:.....	20
Diferencia entre bug e incidencia:	20
Capítulo 4: QA FUNCIONAL Y NO FUNCIONAL	21
QA Funcional	21
QA No Funcional.....	22
Diferencias entre QA Funcional y no Funcional	23
Buenas prácticas para aplicar QA funcional y no funcional	24
Capítulo 5: INTRODUCCIÓN A METODOLOGÍAS ÁGILES (SCRUM, KANBAN)	25
¿Qué son las metodologías ágiles?	26
SCRUM: Estructura y funcionamiento	27
KANBAN: Flujo visual y mejora continua.....	28

Módulo 1: FUNDAMENTOS DEL TESTING DE SOFTWARE



El testing de software, también conocido como prueba de software, es una disciplina clave dentro del ciclo de vida del desarrollo de sistemas, cuya función es evaluar la calidad de una aplicación mediante la identificación de defectos y la verificación del cumplimiento de los requerimientos definidos. Comprender los fundamentos del testing es crucial para asegurar que el software funcione de manera correcta, segura y eficiente antes de ser liberado a producción.

Estos fundamentos se basan en principios como la detección temprana de errores, el reconocimiento de que las pruebas exhaustivas no son posibles en todos los casos, y la importancia de realizar pruebas en etapas tempranas y continuas del desarrollo. Además, se consideran aspectos como la trazabilidad entre los requerimientos y los casos de prueba, la priorización de los riesgos, y la necesidad de validar tanto el funcionamiento técnico como la experiencia del usuario final.

Identificar estos principios proporciona una base sólida para aplicar diversas técnicas de verificación (como las pruebas estáticas) y validación (como las pruebas dinámicas), permitiendo detectar errores tanto en el código como en la lógica del negocio.

En este contexto, el testing no solo se concibe como una etapa final, sino como una práctica integrada y continua a lo largo del desarrollo, con el propósito de mejorar la calidad del producto, reducir costos asociados a fallos en producción, y entregar soluciones confiables que satisfagan las expectativas de los usuarios.

Capítulo 1: ROL DEL QA AUTOMATION EN EQUIPOS ÁGILES

En los entornos de desarrollo ágil, donde la entrega continua de valor al cliente es prioritaria, el rol del QA Automation (Quality Assurance Automation) se ha vuelto fundamental para garantizar la calidad del software sin sacrificar la velocidad. A diferencia de los enfoques tradicionales de aseguramiento de calidad, donde las pruebas se realizaban al final del ciclo de desarrollo, en los equipos ágiles el QA Automation trabaja de manera colaborativa, integrada y continua desde el inicio del proceso.



¿Qué hace un QA Automation en un equipo ágil?

El QA Automation es responsable de diseñar, desarrollar y mantener scripts de prueba automatizados que permiten validar de forma rápida y eficiente el comportamiento del sistema ante distintos escenarios. Su trabajo no se limita a "probar" el software, sino que participa activamente en todas las fases del sprint: desde la planificación hasta la entrega y retrospectiva. Algunas de sus funciones más relevantes incluyen:

- Participar en la definición de criterios de aceptación junto al Product Owner y el equipo de desarrollo.
- Identificar casos de prueba repetitivos que pueden automatizarse.
- Diseñar e implementar pruebas automatizadas (unitarias, de integración, funcionales, regresión, performance).
- Colaborar en la configuración de pipelines de integración y entrega continua (CI/CD).
- Monitorear y mantener las suites de pruebas para asegurar su eficacia y estabilidad.
- Contribuir a la detección temprana de errores mediante pruebas constantes desde las primeras etapas.
- Asegurar que las pruebas estén alineadas con los objetivos del negocio y los requerimientos funcionales y no funcionales.

Beneficios del QA Automation en entornos ágiles

- **Velocidad y eficiencia:** Automatizar pruebas reduce el tiempo necesario para validar funcionalidades nuevas o existentes.
- **Detección temprana de errores:** Permite identificar problemas antes de que escalen, disminuyendo el costo de corrección.
- **Regresión continua:** Garantiza que los cambios no afecten funcionalidades previamente desarrolladas.
- **Consistencia:** Las pruebas automatizadas se ejecutan siempre de la misma forma, evitando errores humanos.
- **Feedback rápido:** Acelera la retroalimentación hacia el equipo de desarrollo, facilitando decisiones informadas.
- **Soporte a DevOps y CI/CD:** Es un pilar fundamental para lograr la automatización total del flujo de trabajo ágil.

Habilidades de un QA Automation en un equipo ágil

Para desempeñar su rol de manera efectiva, un QA Automation debe tener un conjunto amplio de competencias técnicas y blandas, tales como:

- Conocimiento profundo de herramientas de automatización (Selenium, Cypress, Appium, Playwright, TestNG, JUnit, etc.).
- Lenguajes de programación (Java, Python, JavaScript, C#, entre otros).
- Manejo de herramientas de CI/CD como Jenkins, GitLab CI, CircleCI.
- Conocimientos en metodologías ágiles (Scrum, Kanban).
- Habilidades de comunicación y trabajo colaborativo.
- Pensamiento analítico y orientación a la mejora continua.
- Entendimiento de pruebas de software en todos sus niveles (unitarias, integración, sistema, aceptación, performance, seguridad).
- Colaboración con otros roles del equipo ágil

El QA Automation trabaja de la mano con desarrolladores, DevOps, Product Owners y Scrum Masters. No es un rol aislado ni subordinado, sino horizontal y colaborativo, lo que refuerza la idea de que la calidad es responsabilidad de todo el equipo.

Además, la automatización de pruebas ayuda a liberar tiempo a los testers manuales, quienes pueden enfocarse en pruebas exploratorias, pruebas de usabilidad o validaciones más subjetivas que las herramientas automatizadas no pueden cubrir eficientemente.

El rol del QA Automation en equipos ágiles es clave para asegurar que los principios del desarrollo ágil —entrega continua, adaptabilidad al cambio y colaboración constante— se puedan sostener sin comprometer la calidad del software. Este profesional no solo aporta una capa técnica, sino que se convierte en un garante de buenas prácticas, eficiencia y visión a largo plazo en el proceso de calidad. Su integración en el equipo es sinónimo de madurez organizacional y de un enfoque moderno hacia la entrega de soluciones confiables, escalables y alineadas con las necesidades del cliente.



Capítulo 2: TIPOS DE PRUEBAS: UNITARIAS, INTEGRACIÓN, REGRESIÓN, SMOKE, SANITY

En el ámbito del desarrollo de software, las pruebas constituyen un pilar fundamental para garantizar la calidad, estabilidad y confiabilidad de los productos antes de ser entregados a los usuarios finales. A medida que los sistemas se vuelven más complejos y las metodologías ágiles exigen ciclos de entrega más rápidos y frecuentes, se hace imprescindible aplicar diversos tipos de pruebas que aborden distintos niveles y objetivos dentro del proceso de aseguramiento de calidad.

Entre las más relevantes se encuentran las pruebas unitarias, que se enfocan en validar el funcionamiento aislado de componentes individuales del código; las pruebas de integración, que verifican la correcta interacción entre módulos o servicios; las pruebas de regresión, que aseguran que los cambios recientes no afecten funcionalidades ya desarrolladas; y las pruebas smoke y sanity, que permiten realizar validaciones rápidas para determinar si un sistema es lo suficientemente estable o coherente como para continuar con pruebas más exhaustivas.

Cada una de estas pruebas tiene un propósito específico y su adecuada implementación permite detectar errores de forma oportuna, reducir costos de corrección, agilizar los procesos de desarrollo y elevar la confianza en cada nueva versión del software.

Comprender estos tipos de pruebas, su aplicación práctica y su complementariedad dentro del ciclo de vida del software es esencial para lograr productos robustos y alineados con las expectativas del cliente.

1. Pruebas Unitarias (Unit Tests)

Definición:

Son pruebas que validan el comportamiento de componentes individuales del software (funciones, métodos o clases) de forma aislada del resto del sistema.

Objetivo:

Verificar que cada unidad del código funcione correctamente de manera independiente.

Responsables:

Generalmente realizadas por desarrolladores.

Ejemplo:

Probar que una función que suma dos números retorne el valor correcto para diferentes entradas.

Ventajas:

- Detectan errores temprano.
- Facilitan el mantenimiento del código.
- Fomentan el diseño modular.



2. Pruebas de Integración (Integration Tests)

Definición:

Estas pruebas verifican que múltiples unidades de código trabajen correctamente juntas. Evalúan la interacción entre módulos, servicios, bases de datos, APIs, etc.

Objetivo:

Asegurar que los componentes integrados intercambien datos y funciones adecuadamente.

Responsables:

Pueden ser realizadas por desarrolladores o testers.

Ejemplo:

Verificar que un módulo de autenticación se comunique correctamente con la base de datos de usuarios.

Ventajas:

- Identifican problemas de comunicación entre componentes.
- Aseguran consistencia en el flujo de datos.



3. Pruebas de Regresión (Regression Tests)

Definición:

Son pruebas que se realizan después de realizar cambios en el código (como correcciones de bugs o nuevas funcionalidades) para asegurar que las funcionalidades existentes no se hayan visto afectadas.

Objetivo:

Detectar defectos introducidos en versiones nuevas por cambios anteriores.

Responsables:

Generalmente los testers, aunque muchas son automatizadas.

Ejemplo:

Después de modificar el módulo de pagos, se prueba que la funcionalidad de generar facturas siga funcionando correctamente.

Ventajas:

- Previene que errores reaparezcan.
- Aumenta la confiabilidad de cada nueva versión del software.

Pruebas de Regresión



4. Pruebas Smoke (Smoke Tests)

Definición:

Conocidas como “pruebas de humos”, son un conjunto básico de pruebas que se ejecutan rápidamente para asegurar que las funciones esenciales del software están operativas. Se realizan antes de ejecutar pruebas más detalladas.

Objetivo:

Verificar que la aplicación es estable y que puede someterse a pruebas más profundas.

Responsables:

QA testers o automatizadas dentro del pipeline CI/CD.

Ejemplo:

Comprobar que una aplicación web carga correctamente y que el login funciona.

Ventajas:

- Ahorra tiempo al detectar errores críticos temprano.
- Reduce el riesgo de realizar pruebas completas sobre software inestable.



Smoke Testing

5. Pruebas Sanity (Sanity Tests)

Definición:

Son pruebas superficiales orientadas a validar que una funcionalidad específica que ha sido modificada funciona como se espera. Es una forma de verificar "la cordura" del sistema después de pequeñas actualizaciones.

Objetivo:

Confirmar rápidamente que un cambio o corrección no ha afectado otras partes relacionadas.

Responsables:

QA testers.

Ejemplo:

Tras corregir un error en el proceso de envío de correos, se prueba exclusivamente esa funcionalidad para verificar que ahora funcione correctamente.

Ventajas:

- Son rápidas y específicas.
- Aseguran la calidad en ciclos de entrega cortos y frecuentes.



Resumen comparativo

Tipo de prueba	¿Qué prueba?	¿Cuándo se usa?	¿Quién la realiza?
Unitaria	Una unidad de código aislada	Durante el desarrollo	Desarrolladores
Integración	Interacción entre componentes	Después de pruebas unitarias	Desarrolladores/Testers
Regresión	Funcionalidades ya existentes	Tras cambios o nuevas versiones	Testers/Automatización
Smoke	Funcionalidades críticas básicas	Al recibir una nueva build	Testers/Automatización
Sanity	Cambios específicos o correcciones	Después de un cambio puntual	Testers

Capítulo 3: CICLO DE VIDA DEL BUG Y REPORTE DE INCIDENCIAS

En el desarrollo de software, el aseguramiento de la calidad no solo implica la detección de errores, sino también su adecuada documentación, seguimiento y resolución. Para lograrlo de manera estructurada y eficiente, es fundamental comprender el ciclo de vida del bug, también conocido como ciclo de vida de una incidencia, el cual describe las distintas fases por las que transita un defecto desde que es identificado hasta que se soluciona y se cierra formalmente.

Este proceso no solo permite mantener la trazabilidad y el control de los errores, sino que también facilita la comunicación entre los distintos roles del equipo —desarrolladores, testers, líderes de proyecto, product owners, entre otros— y contribuye a la mejora continua del producto. Junto a esto, el reporte de incidencias se convierte en una herramienta clave, ya que permite registrar de forma clara, detallada y sistemática cualquier desviación respecto del comportamiento esperado del sistema, proporcionando la información necesaria para su análisis y resolución.

La calidad del reporte influye directamente en la eficiencia del ciclo de corrección, por lo que su elaboración debe seguir estándares y buenas prácticas bien definidos. Conocer en profundidad el ciclo de vida de los bugs y dominar el arte de reportar incidencias no solo optimiza el trabajo del equipo de QA, sino que también garantiza que los defectos sean corregidos oportunamente, asegurando entregas más robustas y satisfactorias para el cliente.

Ciclo de vida del Bug (Bug Life Cycle)

El ciclo de vida de un bug describe el recorrido que sigue un defecto desde su detección hasta su cierre. Cada etapa representa un estado del bug que permite entender en qué punto se encuentra dentro del flujo de trabajo del equipo de desarrollo y aseguramiento de calidad.

Etapas del ciclo de vida de un bug:

- **New (Nuevo):** El bug ha sido detectado y reportado por un tester o usuario. Aún no ha sido evaluado.
- **Assigned (Asignado):** El bug ha sido revisado por el líder del equipo o el encargado de QA y se asigna a un desarrollador específico para su resolución.
- **Open (Abierto):** El desarrollador confirma la validez del bug y comienza a trabajar en la solución.
- **In Progress (En progreso):** El desarrollador está trabajando activamente en la corrección del bug.
- **Fixed (Corregido):** El desarrollador indica que el bug ha sido solucionado y sube el cambio al entorno correspondiente.
- **Testing (En pruebas / Ready for Testing):** El tester verifica que la corrección sea efectiva y que no se hayan introducido nuevos errores.
- **Reopened (Reabierto):** Si el tester comprueba que el problema persiste o que la corrección generó nuevos fallos, el bug vuelve al estado "Open".
- **Verified (Verificado):** El tester valida que el bug ha sido solucionado correctamente en todos los entornos necesarios.
- **Closed (Cerrado):** El bug se da por solucionado de forma definitiva y se cierra oficialmente.
- **Rejected / Invalid / Duplicate / Not a Bug:** El bug se marca como inválido si no representa un error real, si es duplicado de otro ya reportado o si no requiere corrección.



Las etapas pueden variar ligeramente según las herramientas o procesos de cada equipo.

Reporte de Incidencias

El reporte de incidencias es el documento o registro mediante el cual se reporta un error o comportamiento inesperado del sistema. Su calidad es esencial para que el equipo de desarrollo pueda comprender y reproducir el problema.

Elementos de un buen reporte de incidencia:

- **Identificador único:** Código o número automático generado por el sistema de seguimiento (bug tracker).
- **Título breve y descriptivo:** Debe sintetizar claramente el problema (ej. "Error al guardar formulario de contacto en modo offline").
- **Descripción detallada:** Explica el error, qué se esperaba y qué ocurrió realmente.
- **Pasos para reproducir:** Instrucciones precisas y numeradas que permitan recrear el bug.
- **Resultado esperado vs. resultado obtenido:** Clarifica la discrepancia entre lo que debería suceder y lo que efectivamente ocurrió.
- **Evidencias:** Capturas de pantalla, videos o logs del sistema que respalden el reporte.
- **Severidad y prioridad:**
 - ✓ **Severidad:** impacto técnico del bug (crítico, mayor, menor, etc.).
 - ✓ **Prioridad:** urgencia con la que debe resolverse según el negocio.
- **Entorno de pruebas:** Información sobre el sistema operativo, navegador, versión del software, etc.
- **Estado actual del bug:** De acuerdo con las etapas del ciclo de vida.

Herramientas comunes para gestión de bugs e incidencias:



Jira

Amplia funcionalidad para equipos ágiles, integración con Scrum y Kanban.



Bugzilla

Enfocado exclusivamente en el rastreo de bugs, muy detallado y personalizable.



Redmine

Open source, ideal para la gestión de proyectos y seguimiento de errores.



MantisBT

Ligero y sencillo para equipos pequeños y medianos.



GitHub Issues / GitLab Issues

Muy útiles en proyectos de desarrollo colaborativo con control de versiones.

Buenas prácticas en el reporte de incidencias:

- Redactar de forma objetiva y profesional, evitando suposiciones.
- Ser conciso pero completo: ni omitir información ni extenderse innecesariamente.
- Verificar que el bug no haya sido reportado previamente.
- Reproducir el bug varias veces antes de reportarlo.
- Clasificar correctamente la severidad y prioridad.
- Revisar la redacción antes de enviarlo, para evitar ambigüedades.

Diferencia entre bug e incidencia:

- **Bug:** es un error técnico o defecto del sistema que genera un comportamiento incorrecto.
- **Incidencia:** es un término más general que puede referirse a cualquier desviación del comportamiento esperado, incluyendo bugs, errores de configuración, fallos del entorno o requerimientos mal entendidos.

El correcto manejo del ciclo de vida del bug y el reporte de incidencias no solo mejora la eficiencia del equipo de desarrollo, sino que también permite construir software más robusto, estable y alineado con los requerimientos del usuario.

Es una práctica esencial dentro de la cultura de calidad y mejora continua, y debe ser integrada como parte natural del flujo de trabajo ágil.

Capítulo 4: QA FUNCIONAL Y NO FUNCIONAL

En el mundo del aseguramiento de la calidad (QA, por sus siglas en inglés), es esencial comprender que no todos los aspectos de un sistema pueden evaluarse con los mismos criterios. Para garantizar que una aplicación cumpla con las expectativas del cliente y los estándares del negocio, se requiere un enfoque que contemple tanto las funcionalidades del sistema como sus características no funcionales.

Es en este contexto donde se distinguen dos grandes ramas dentro del proceso de testing: el QA funcional y el QA no funcional. Ambos son complementarios y necesarios, y su correcta aplicación permite verificar, por un lado, que el sistema haga “lo que debe hacer” (funcional) y, por otro, “cómo lo hace” (no funcional).

Esta división permite diseñar estrategias de prueba más eficaces, priorizar los esfuerzos de validación y detectar errores que podrían comprometer la usabilidad, el rendimiento, la seguridad o la estabilidad de una aplicación. Dominar las diferencias entre ambos enfoques y saber aplicarlos en el momento y contexto adecuados es clave para lograr un software robusto, confiable y competitivo.

QA Funcional

¿Qué es?

El QA funcional se enfoca en probar las funcionalidades del sistema conforme a los requerimientos definidos. Busca verificar qué hace el sistema y si su comportamiento cumple con las expectativas del usuario final.

Objetivos:

- Asegurar que todas las funciones del sistema operen como se especificó.
- Verificar flujos de trabajo, validaciones de formularios, interacciones con bases de datos, lógica de negocio, etc.

Tipos de pruebas funcionales:

- **Pruebas unitarias:** validan funciones individuales.
- **Pruebas de integración:** validan la interacción entre módulos.
- **Pruebas de sistema:** evalúan el sistema completo.
- **Pruebas de aceptación del usuario (UAT):** validación final por parte del cliente.

Ejemplos:

- Verificar que un botón “Guardar” almacene correctamente los datos.
- Confirmar que un usuario no pueda ingresar con credenciales incorrectas.
- Validar que una factura se genere automáticamente tras una venta.

QA No Funcional

¿Qué es?

El QA no funcional se enfoca en evaluar cómo se comporta el sistema bajo ciertas condiciones, como carga, seguridad, velocidad de respuesta, usabilidad, escalabilidad, entre otros. No se centra en las funcionalidades, sino en atributos de calidad.

Objetivos:

- Medir el rendimiento del sistema bajo distintas cargas.
- Validar su seguridad ante accesos indebidos.
- Evaluar la facilidad de uso, accesibilidad, capacidad de recuperación, etc.

Tipos de pruebas no funcionales:

- **Pruebas de rendimiento:** velocidad, capacidad de respuesta.
- **Pruebas de carga y estrés:** comportamiento bajo uso intensivo o límites.
- **Pruebas de seguridad:** protección de datos, control de accesos.
- **Pruebas de usabilidad:** experiencia de usuario.
- **Pruebas de compatibilidad:** funcionamiento en distintos navegadores, dispositivos o sistemas operativos.
- **Pruebas de escalabilidad y disponibilidad.**

Ejemplos:

- Medir cuánto tarda en cargar una página con 1000 usuarios concurrentes.
- Verificar que los datos estén encriptados en una transferencia.
- Comprobar que una aplicación sea accesible para personas con discapacidad visual.

Diferencias entre QA Funcional y no Funcional

Características	QA Funcional	QA No Funcional
Enfoque	Qué hace el sistema	Cómo se comporta el sistema
Basado en	Requisitos funcionales	Requisitos no funcionales
Objetivo	Validar acciones, procesos y lógica	Validar rendimiento, seguridad, experiencia
Ejemplos	Login, pagos, búsquedas, formularios	Velocidad, carga, accesibilidad, fallos
Herramientas comunes	Selenium, Postman, SoapUI	Jmeter, LoadRunner, OWASP ZAP, Lighthouse

Buenas prácticas para aplicar QA funcional y no funcional

- Analizar requerimientos desde ambas perspectivas desde el inicio del proyecto.
- Automatizar pruebas funcionales repetitivas y pruebas no funcionales críticas como carga y rendimiento.
- Incluir pruebas no funcionales desde etapas tempranas, no solo al final del desarrollo.
- Trabajar en conjunto con desarrollo, UX y DevOps para cubrir todas las áreas del producto.
- Priorizar pruebas según el impacto en el usuario final y el riesgo del negocio.
- Usar herramientas especializadas para cada tipo de prueba, combinando tests manuales y automatizados.

El aseguramiento de la calidad debe entenderse como un enfoque holístico que considere no solo si el sistema realiza correctamente sus funciones, sino también si lo hace con la eficiencia, seguridad y experiencia que los usuarios esperan.

El equilibrio entre QA funcional y no funcional es la base de un producto de calidad. Implementarlos adecuadamente mejora la satisfacción del cliente, reduce errores en producción y fortalece la competitividad del producto en el mercado.

Capítulo 5: INTRODUCCIÓN A METODOLOGÍAS ÁGILES (SCRUM, KANBAN)

En el dinámico y competitivo mundo del desarrollo de software y la gestión de proyectos, la capacidad de adaptarse al cambio, entregar valor continuo y trabajar en equipo de forma colaborativa se ha vuelto esencial. Es en este contexto donde emergen con fuerza las metodologías ágiles, un conjunto de principios y prácticas que promueven la flexibilidad, la entrega incremental y la mejora continua.

A diferencia de los enfoques tradicionales como el modelo en cascada, las metodologías ágiles priorizan la interacción constante con el cliente, la adaptación rápida a los cambios de requerimientos y la colaboración entre todos los miembros del equipo.

Entre las metodologías ágiles más utilizadas destacan Scrum y Kanban, cada una con sus propias características, roles y herramientas, pero compartiendo el objetivo común de optimizar los procesos, reducir tiempos y aumentar la calidad del producto final.

Comprender las bases de estas metodologías es clave para cualquier profesional que busque participar de equipos de trabajo modernos, especialmente en entornos tecnológicos o de innovación continua.



¿Qué son las metodologías ágiles?

Las metodologías ágiles son marcos de trabajo que se basan en los valores y principios del Manifiesto Ágil (2001), que enfatiza:

- Individuos e interacciones por sobre procesos y herramientas.
- Software funcionando por sobre documentación extensiva.
- Colaboración con el cliente por sobre negociación de contratos.
- Respuesta al cambio por sobre seguimiento de un plan.

Estas metodologías buscan entregar productos funcionales en ciclos cortos, fomentando la mejora constante, la entrega frecuente de valor, la comunicación fluida y la autonomía de los equipos.

ón del
durante

KANBAN: Flujo visual y mejora continua

Kanban es un enfoque ágil enfocado en la visualización del flujo de trabajo, la limitación del trabajo en curso (WIP) y la optimización del proceso continuo. Se representa generalmente mediante un tablero Kanban que muestra el progreso de las tareas desde el inicio hasta su finalización.

Elementos:

- **Tablero Kanban:** Se divide en columnas que representan etapas del proceso (Ej. "Por hacer", "En progreso", "En revisión", "Hecho").
- **Tarjetas:** Cada tarjeta representa una tarea o ítem de trabajo.
- **Límites WIP:** Se establece un número máximo de tareas permitidas en cada etapa para evitar sobrecarga y mejorar el flujo.
- **Políticas explícitas:** Reglas claras para mover tareas entre columnas.
- **Métricas:** Se usan indicadores como lead time, throughput y tiempo de ciclo para evaluar y mejorar.

Beneficios:

- Alta visibilidad y control del trabajo.
- Reducción de tiempos de entrega.
- Mejora continua sin interrupciones.
- Ideal para equipos con trabajos variables o de soporte.

